

Prova 2 (versão: 19/7/23)

Disciplina: Complexidade e Projeto de Algoritmos

Professor: Vinícius Dias (viniciusdias@ufla.br)

1 Considere o problema da mochila: dados n itens, pesos dos itens $w_1, w_2, \dots, w_n$, valores dos itens $v_1, v_2, \dots, v_n$ e uma capacidade máxima de peso, o objetivo é escolher um subconjunto de itens que maximize a soma dos valores dos itens ao mesmo tempo que respeite a restrição de peso máximo da mochila W . Considere o seguinte algoritmo de programação dinâmica que encontra o custo de uma solução ótima para o problema da mochila:

```
KNAPSACK-PD( $n, w, v, W$ )  
 $c \leftarrow$  arranjo bidimensional  $(n + 1) \times (W + 1)$   
for  $i \leftarrow 0$  to  $n$   
   $c[i, 0] \leftarrow 0$   
for  $i \leftarrow 1$  to  $W$   
   $c[0, i] \leftarrow 0$   
for  $i \leftarrow 1$  to  $n$   
  for  $j \leftarrow 1$  to  $W$   
    if  $w_i > j$   
       $c[i, j] \leftarrow c[i - 1, j]$   
    else  
       $c[i, j] \leftarrow \max(c[i - 1, j], c[i - 1, j - w_i] + v_i)$   
return  $c[n, W]$ 
```

- a. Execute passo-a-passo o algoritmo da mochila para a instância a seguir. Você deve mostrar a estrutura de dados e a ordem em que ela foi preenchida, considerando apenas os custos das soluções.

	$n = 5; W = 7$				
i	1	2	3	4	5
v_i	2	1	3	4	1
w_i	4	2	1	2	2

- b. Indique como modificar o algoritmo de forma que ele seja capaz de informar, além dos custos ótimos, os itens selecionados que geram esse custo. Explique como utilizar essa informação para recuperar os itens da solução.
- c. Seja o procedimento não-exato para o problema que escolhe sempre o próximo item de melhor custo benefício (maior v_i/w_i) até que isso não seja mais possível. Execute o procedimento para a entrada do item (a). O que você pode (ou não) concluir a partir do resultado?

2 Problema do robô coletor. Um ambiente de dimensões $n \times m$ (pense em um tabuleiro) está repleto de itens espalhados, em posições arbitrárias e fixas. Seu objetivo é coletar a maior quantidade de itens do ambiente e para isso, você pode contar com um robô coletor. Ele só é capaz de andar em duas direções e não tem nenhum tipo de sensor que detecta a direção onde tem itens, isto é, sua rota precisa ser pré-programada. Vamos considerar que esse robô é posicionado no canto inferior esquerdo da sala e que ele consegue tomar duas direções: para cima ou para direita. Além disso, toda vez que esse robô passa por cima de uma posição que contenha um item, ele o coleta automaticamente, podendo continuar sua trajetória. Objetivo do problema: receber a descrição da sala e de onde os itens estão

(uma matriz $n \times m$ formada por 0, quando a posição estiver vazia ou 1, quando existir item na posição), e determinar uma rota ótima para o robô, isto é, aquela onde ele colete mais itens. A figura a seguir ilustra o problema com um exemplo.

	1	2	3	4
1		item		
2				item
3		item	item	
4				
5	robô			item

- Apresente a recorrência que representa o custo das soluções ótimas para os subproblemas, de forma que possa ser usado para construir um algoritmo de programação dinâmica (não precisa escrever o algoritmo, apenas a recorrência de custo).

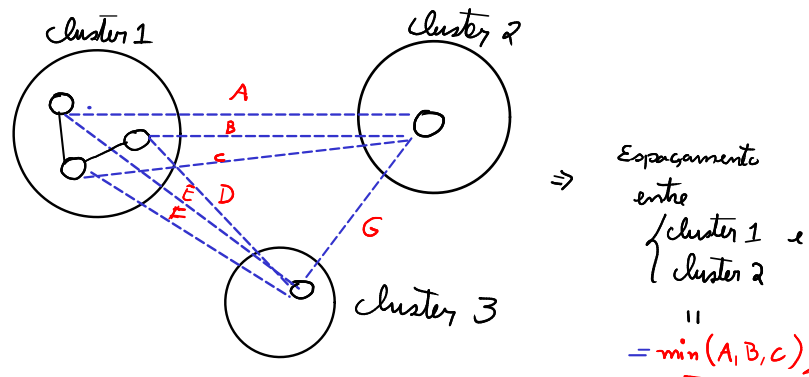
3 Mostrar que:

- O problema da coloração de grafos está em NP. O problema é definido como: dados um grafo $G(V, E)$ e uma quantidade de cores k , determinar se é possível atribuir uma das k cores a cada vértice de maneira que nenhum par de vértices adjacentes compartilhe a mesma cor.
- O problema da mediana está em P. O problema é definido como: dados um arranjo de números $a[1..n]$ e um número m , determinar se m é a mediana dos números em a .
- O problema da cobertura por vértices está em NPC, sabendo que: (1) o problema da cobertura por vértices está em NP; (2) o problema da clique está em NPC. O problema da cobertura por vértices é definido como: dados um grafo $G(V, E)$ e um inteiro k , determinar se existe um subconjunto de vértices V' de tamanho k de tal forma que toda aresta do grafo seja incidente a algum vértice em V' . O problema da clique é definido como: dados um grafo $G(V, E)$ e um inteiro k , determinar se o grafo contém uma clique de k vértices.

4 O problema do agrupamento (*clustering*)¹ é muito utilizado em aprendizado de máquina e mineração de dados para organizar itens (pessoas em redes sociais, padrões de trajetória geolocalizados, estruturas biológicas similares entre si, etc.) em k grupos. Naturalmente, existem diversas funções de otimização que podem ser utilizadas como critério de agrupamento dos itens. Um desses critérios procura maximizar o espaçamento entre os k grupos: isto é, **maximizar o espaçamento mínimo entre os pares de grupos da solução**. Você pode assumir que os itens são pontos em um espaço multidimensional e que a distância entre 2 pontos p_1 e p_2 é determinada pela função $d(p_1, p_2)$. O espaçamento entre 2 grupos é definido como a distância mínima entre qualquer par de pontos de grupos diferentes.

¹o termo em inglês *cluster* é usado como sinônimo para grupo.

A figura a seguir mostra um exemplo visual que ajuda a entender o problema. **Entrada:** Conjunto de n pontos $P = \{p_1, p_2, \dots, p_n\}$; função de distância $d : (P, P) \rightarrow \mathbb{N}$ usada para determinar a distância entre dois pontos; inteiro $k \geq n$ representando a quantidade de grupos pretendida. **Saída:** k conjuntos $G = \{G_1, G_2, \dots, G_k\}$ representando k grupos que maximizam o espaçamento entre grupos.



Objetivo: maximizar o espaçamento mínimo entre os clusters

Para o exemplo acima:

cluster	cluster	espaçamento
1	2	$\min(A, B, C)$
1	3	$\min(D, E, F)$
2	3	$\min(G) = G$

O mínimo entre esses mínimos é o custo da solução do exemplo.

Você quer maximizar o mínimo de mínimos.

- Explique como usar o algoritmo de Kruskal para produzir os k grupos com espaçamento maximizado. Dica: pense na relação entre esse problema e o funcionamento do algoritmo de Kruskal para árvores geradoras mínimas. Forneça o algoritmo como descrição textual precisa (ou pseudocódigo, se preferir).